



算法分析与设计

📅 发表于 2025-12-29 | ⌚ 更新于 2025-12-29 | 📁 算法
| 👁 浏览量: 5

第一章 算法概述

五种算法渐进界

1. O (Big-O) —— 渐近上界

设 $f(n)$ 和 $g(n)$ 是定义在非负整数上的正函数。如果存在正常数 c 和 n_0 , 使得对于所有的 $n \geq n_0$, 都有:

$$0 \leq f(n) \leq c \cdot g(n)$$

则称 $f(n) = O(g(n))$ 。

2. Ω (Big-Omega) —— 渐近下界

设 $f(n)$ 和 $g(n)$ 是定义在非负整数上的正函数。如果存在正的常数 c 和 n_0 , 使得对于所有的 $n \geq n_0$, 都有:

$$0 \leq c \cdot g(n) \leq f(n)$$

则称 $f(n) = \Omega(g(n))$ 。

3. Θ (Big-Theta) —— 渐近紧确界

设 $f(n)$ 和 $g(n)$ 是定义在非负整数上的正函数。如果存在正的常数 c_1 , c_2 和 n_0 , 使得对于所有的 $n \geq n_0$, 都有:

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

则称 $f(n) = \Theta(g(n))$ 。

4. o (Little-o) —— 非紧确上界

设 $f(n)$ 和 $g(n)$ 是定义在非负整数上的正函数。如果对于任意正的常数 $c > 0$, 都存在 n_0 , 使得对于所有的 $n \geq n_0$, 都有:

$$0 \leq f(n) < c \cdot g(n)$$

则称 $f(n) = o(g(n))$ 。(等价定义: $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$), 通常证明时使用极限

5. ω (Little-omega) —— 非紧确下界

设 $f(n)$ 和 $g(n)$ 是定义在非负整数上的正函数。如果对于任意正的常数 $c > 0$, 都存在 n_0 , 使得对于所有的 $n \geq n_0$, 都有:

$$0 \leq c \cdot g(n) < f(n)$$

则称 $f(n) = \omega(g(n))$ 。(等价定义: $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$), 通常证明时使用极限

解递归方程

在算法分析中, 递归方程主要分为两类:

1. **线性递推式** (如斐波那契数列): $a_n = c_1 a_{n-1} + c_2 a_{n-2} + \dots$
2. **分治递归式** (如归并排序): $T(n) = aT(n/b) + f(n)$

一、常系数线性齐次递推方程

形式: $a_n + c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_k a_{n-k} = 0$

解法步骤（特征方程法）：

1. 写出特征方程：将 a_n 替换为 x^k , a_{n-1} 替换为 x^{k-1} , 以此类推。例如对于二阶方程 $a_n - c_1a_{n-1} - c_2a_{n-2} = 0$, 特征方程为：
$$x^2 - c_1x - c_2 = 0$$
2. 求特征根：解出方程的根 $r_1, r_2, \dots$ 。
3. 写出通解：根据根的情况，通解形式如下（以二阶为例）：

根的情况	通解公式 (A,B 为待定常数)
两个不相等的实根($r_1 \neq r_2$)	$a_n = A \cdot r_1^n + B \cdot r_2^n$
两个相等的实根($r_1 = r_2 = r$)	$a_n = (A + B \cdot n) \cdot r^n$

4. 求特解（确定常数）：代入初始条件（如 a_0, a_1 的值），解方程组求出 A 和 B 。

二、常系数线性非齐次递推方程

形式： $a_n + c_1a_{n-1} + \dots = f(n)$

(等式右边不是0，而是一个关于 n 的函数)

解法步骤：

通解 = 齐次通解 + 特解 ($a_n = a_n^{(h)} + a_n^{(p)}$)

1. 求齐次通解 $a_n^{(h)}$ ：令右边 $f(n) = 0$, 按照第一部分的方法求出通解。
2. 求特解 $a_n^{(p)}$ ：根据 $f(n)$ 的形式，“猜测”特解的形式并代入原方程求解系数。

3.

f(n) 的形式	猜测特解 $a_n^{(p)}$ 的形式	注意事项
多项式 $P_k(n)$ (如 $n^2 + 1$)	$Q_k(n)$ (同阶多项式, 如 $An^2 + Bn + C$)	如果 1 是特征根, 需乘 n 或 n^2

f(n) 的形式	猜测特解 an(p) 的形式	注意事项
指数函数 β^n (如 3^n)	$C \cdot \beta^n$	如果 β 是特征根，需乘 n (重根乘 n^2)

4. 组合并求解： 将 $a_n^{(h)} + a_n^{(p)}$ 组合，代入初始条件求解常数。

三、分治递归方程 (Divide and Conquer)

形式： $T(n) = aT(n/b) + f(n)$

(常用于分析归并排序、二分搜索等算法的时间复杂度)

方法1：主定理 (Master Theorem) —— 最常用

比较 $n^{\log_b a}$ 与 $f(n)$ 的增长速率：

- 1. 情况 1 (根重)： 如果 $f(n) = O(n^{\log_b a - \epsilon})$ (即 $f(n)$ 增长慢) ，
则： $T(n) = \Theta(n^{\log_b a})$
- 2. 情况 2 (平衡)： 如果 $f(n) = \Theta(n^{\log_b a})$ (即同阶) ， 则： $T(n) = \Theta(n^{\log_b a} \lg n)$
- 3. 情况 3 (叶重)： 如果 $f(n) = \Omega(n^{\log_b a + \epsilon})$ (即 $f(n)$ 增长快) ， 且
满足正则条件， 则： $T(n) = \Theta(f(n))$

方法2：递归树法 (Recursion Tree)

当主定理不适用或需要画图分析时使用。

- 1. 画出树状结构， 每层写出消耗的代价。
- 2. 对每一层的代价求和。
- 3. 对所有层求总和。

方法3：代换法 (Substitution Method)

- 1. **猜测解的形式** (如猜测 $T(n) = O(n \lg n)$) 。
- 2. 使用数学归纳法**证明**猜测是正确的。

示例：解 $a_n = 5a_{n-1} - 6a_{n-2}, a_0 = 1, a_1 = 0$

1. 特征方程: $x^2 - 5x + 6 = 0$
 2. 求根: $(x - 2)(x - 3) = 0 \Rightarrow r_1 = 2, r_2 = 3$
 3. 通解: $a_n = A \cdot 2^n + B \cdot 3^n$
 4. 代入初始条件:
 - $n = 0 : A + B = 1$
 - $n = 1 : 2A + 3B = 0$
 5. 解方程组: 由(1)得 $A = 1 - B$, 代入(2): $2(1 - B) + 3B = 0$
 $\Rightarrow 2 - 2B + 3B = 0 \Rightarrow B = -2$ 则 $A = 1 - (-2) = 3$
 6. 最终解: $a_n = 3 \cdot 2^n - 2 \cdot 3^n$
-

第二章 递归与分治

二分搜索

```
1  int binarysearch(int a[ ],int x,int n){
2
3      int right = n-1,left = 0;
4      while(left<=right){
5          int mid = (right + left) / 2;
6          if(a[mid]==x) return mid;
7          if(a[mid]>x) right = mid-1;
8          else left = mid+1;
9      }
10     return -1;
11
12 }
```

每执行一次算法的 while 循环, 待搜索数组的大小减少一半。因此, 在最坏情况下, while 循环被执行了 $O(\log n)$ 次。循环体内运算需要 $O(1)$ 时间, 因此整个算法在最坏情况下的计算时间复杂度为 $O(\log n)$ 。

合并排序

实现代码

```
1 // 辅助函数：合并两个有序子数组
2 // 将 array[left...mid] 和 array[mid+1...right] 合并
3 void Merge(int array[], int left, int mid, int right) {
4     // 1. 创建临时数组并复制数据
5     int n1 = mid - left + 1;
6     int n2 = right - mid;
7     int L[n1], R[n2]; // 左右临时数组
8
9     for (int i = 0; i < n1; i++) L[i] = array[left + i];
10    for (int j = 0; j < n2; j++) R[j] = array[mid + 1 + j];
11
12    // 2. 归并过程
13    int i = 0; // L的索引
14    int j = 0; // R的索引
15    int k = left; // 原数组索引
16
17    while (i < n1 && j < n2) {
18        // 比较两边的首元素，取较小的放入原数组
19        if (L[i] <= R[j]) {
20            array[k] = L[i];
21            i++;
22        } else {
23            array[k] = R[j];
24            j++;
25        }
26        k++;
27    }
28
29    // 3. 处理剩余元素
30    while (i < n1) { // 如果左边有剩
31        array[k] = L[i];
32        i++; k++;
33    }
34    while (j < n2) { // 如果右边有剩
35        array[k] = R[j];
36        j++; k++;
37    }
38 }
39
40 // 主排序函数
41 void MergeSort(int array[], int left, int right) {
42     if (left < right) {
43         // 1. 找中间点
44         int mid = (left + right) / 2;
45
46         // 2. 递归排序左半部分
```

```

47         MergeSort(array, left, mid);
48
49         // 3. 递归排序右半部分
50         MergeSort(array, mid + 1, right);
51
52         // 4. 合并
53         Merge(array, left, mid, right);
54     }
55 }

```

非递归版

```

1
2 // 1. 合并函数 (逻辑与递归版完全一样)
3 // 将 arr[left...mid] 和 arr[mid+1...right] 合并
4 void Merge(vector<int>& arr, int left, int mid, int right) {
5     int n1 = mid - left + 1;
6     int n2 = right - mid;
7
8     // 创建临时数组
9     vector<int> L(n1), R(n2);
10
11     for (int i = 0; i < n1; i++) L[i] = arr[left + i];
12     for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
13
14     int i = 0, j = 0, k = left;
15     while (i < n1 && j < n2) {
16         if (L[i] <= R[j]) {
17             arr[k++] = L[i++];
18         } else {
19             arr[k++] = R[j++];
20         }
21     }
22
23     while (i < n1) arr[k++] = L[i++];
24     while (j < n2) arr[k++] = R[j++];
25 }
26
27 // 2. 非递归合并排序 (主函数)
28 void MergeSortNonRecursive(vector<int>& arr) {
29     int n = arr.size();
30     if (n <= 1) return;
31
32     // curr_size 表示当前正在合并的子数组的大小
33     // 变化过程: 1 -> 2 -> 4 -> 8 ...
34     for (int curr_size = 1; curr_size < n; curr_si

```

```

35
36         // left_start 是每一对子数组的起始位置
37         // 每次跳跃 2 * curr_size
38         for (int left_start = 0; left_start < n -
39
40             // 找到中间点，注意不要越界
41             int mid = min(left_start + curr_size -
42
43             // 找到右边界，注意不要越界
44             int right_end = min(left_start + 2 * c
45
46             // 只有当存在右半部分时才需要合并
47             // 如果 mid 已经是最后一个元素，说明没有右半部分
48             if (mid < right_end) {
49                 Merge(arr, left_start, mid, right_end);
50             }
51         }
52     }
53 }

```

时间复杂度

$$T(n) = 2T(n/2) + O(n)$$

使用主定理分析：

$$a = 2, b = 2, f(n) = n$$

计算关键指数：

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

由于 $f(n) = \Theta(n^{\log_b a})$ ，符合主定理第二种情况，因此：

$$T(n) = O(n \log n)$$

快速排序

```

1 // 核心分区函数：返回基准值的最终索引
2 int partition(int arr[], int left, int right) {
3
4     // 生成 [left, right] 之间的随机索引
5     int randIdx = left + rand() % (right - left + 1);
6     swap(arr[left], arr[randIdx]); // C++ 内置 swap

```

```
7
8     int pivot = arr[left]; // 1. 选最左边为基准，挖坑
9
10    while (left < right) {
11        // 2. 从右向左找比 pivot 小的数
12        while (left < right && arr[right] >= pivot)
13            right--;
14    }
15    arr[left] = arr[right]; // 填左边的坑，right 左移
16
17    // 3. 从左向右找比 pivot 大的数
18    while (left < right && arr[left] <= pivot)
19        left++;
20    }
21    arr[right] = arr[left]; // 填右边的坑，left 右移
22 }
23
24 // 4. 基准值填入最后的坑
25 arr[left] = pivot;
26 return left;
27 }
28
29 //教材写法，交换法
30 int Partition(int a[], int l, int r) {
31     int i = l, j = r+1;
32     int x = a[l];
33     while (true) {
34         while (a[++i] < x && i < r);
35         while (a[--j] > x);
36         if (i >= j) break;
37         Swap(a[i], a[j]);
38     }
39     a[l] = a[j];
40     a[j] = x;
41     return j;
42 }
43
44 // 主排序函数
45 void quicksort(int arr[], int left, int right) {
46     //可以加上两步，从左往右扫描数组，如果是升序直接返回
47     //TODO step 1
48     //TODO step 2
49     if (left < right) {
50         // 分区：此时 pivotIdx 位置的元素已经排好了
51         int pivotIdx = partition(arr, left, right)
52     }
```

```
53         // 递归排序左边
54         quicksort(arr, left, pivotIdx - 1);
55
56         // 递归排序右边
57         quicksort(arr, pivotIdx + 1, right);
58     }
59 }
60
```

快速排序 (Quick Sort) 的时间复杂度分析完全取决于**划分 (Partition) 的质量**，也就是基准值 (Pivot) 选得好不好。

以下是关于最好情况和最坏情况的详细分析：

1. 最好情况 (Best Case)

条件： 每次划分选取的基准值 (Pivot) 恰好都是当前子数组的**中位数**。这意味着每次都能将数组**均匀地**一分为二，左边一半，右边一半。

分析推导：

1. **递归结构：** 这时的递归树是一棵标准的、平衡的二叉树。

2. **递推公式：**

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

- $2T(n/2)$ ：解决左右两个规模减半的子问题。
- $O(n)$ ：本层划分 (Partition) 所需的遍历比较时间。

3. **求解：** 根据主定理 (Master Theorem) 或递归树法：

- 递归深度为 $\log_2 n$ 。
- 每一层的工作量总和是 $O(n)$ 。
- 总时间 = 层数 × 每层工作量。

结论：

$$T(n) = O(n \log n)$$

2. 最坏情况 (Worst Case)

条件： 每次划分选取的基准值都是当前子数组中的**最大值或最小值**。

这种情况通常发生在：

- 1. 数组**已经有序**（或**逆序**）。
- 2. 并且我们固定选取**第一个元素**（或**最后一个**）作为基准值。

分析推导：

- 1. **划分极度不平衡：** 比如数组 $[1, 2, 3, 4, 5]$ ，选 1 做基准。
 - 左边子数组长度： 0
 - 右边子数组长度： $n - 1$
 - 这导致递归树退化成了一个**链表**（斜树）。

2. **递推公式：**

$$T(n) = T(n - 1) + T(0) + O(n) = T(n - 1) + O(n)$$

- $T(n - 1)$ ：只解决了一个规模减少 1 的子问题。
- $O(n)$ ：本层划分的时间。

3. **求解（等差数列求和）：**

$$\begin{aligned} T(n) &= n + (n - 1) + (n - 2) + \cdots + 1 \\ &= \frac{n(n + 1)}{2} \\ &\approx \frac{n^2}{2} \end{aligned}$$

结论：

$$T(n) = O(n^2)$$

3. 总结对比图表

情况	发生条件	递归树形态	递推公式	时间复杂度
最好情况	每次选到中位数	平衡二叉树	$T(n) = 2T(n/2) + n$	$O(n\log n)$

情况	发生条件	递归 树形 态	递推公式	时间复杂度
最坏情 况	每次选到极 值 (有序数 组)	线性 链表 (斜 树)	$T(n) = T(n - 1) + n$	$O(n^2)$
平均情 况	随机选取基 准	近似 平衡	N/A	$O(n \log n)$

4. 如何避免最坏情况？

这就回到了我们之前讨论的代码优化：

- **随机化 (Randomization)**：在 Partition 之前，随机选一个数与 left 交换。这样即使原数组有序，也能打破其顺序，将期望复杂度稳定在 $O(n \log n)$ 。
- **三数取中 (Median of Three)**：取头、中、尾三个数的中位数作为基准，也能有效避免有序数组带来的 $O(n^2)$ 问题。

线性时间选择

找出第k小的元素

随机化选择

```
1  int RandomizedSelect(int a[], int l, int r, int k)
2      if (l == r) return a[l];
3      int i = RandomizedPartition(a, l, r); // 使用随机
4      int j = i - l + 1; // 计算基准是第几小
5      if (k <= j)
6          return RandomizedSelect(a, l, i, k);
7      else
8          return RandomizedSelect(a, i + 1, r, k - j)
9  }
```

线性时间选择：

```

1  int Select(int a[], int l, int r, int k) {
2      if (r - l < 75) {
3          // 小规模数据直接用某个简单算法排序返回
4          // insertSort(a, l, r);
5          return a[l + k - 1];
6      }
7
8      // 1. 分组：每5个一组，找中位数，换到数组前部
9      for (int i = 0; i <= (r - l - 4) / 5; i++) {
10         int s = l + 5 * i;
11         int t = s + 4;
12         for (int j = 0; j < 3; j++) bubble(s, t -
13             swap(a[l + i], a[s + 2])); // 第3小即中位数
14     }
15
16     // 2. 递归找到“中位数的中位数” x
17     int x = Select(a, l, l + (r - l - 4) / 5, (r -
18
19     // 3. 以 x 为基准进行划分
20     int i = Partition(a, l, r, x);
21     int j = i - l + 1;
22
23     // 4. 判断 k 落在哪个区间
24     if (k <= j) return Select(a, l, i, k);
25     else return Select(a, i + 1, r, k - j);
26 }

```

时间复杂度分析

证明 x 至少比 $\frac{3n}{10} - 1$ 个元素大

- 第一步：分组与中位数 我们将 n 个元素分成 $m = \lfloor n/5 \rfloor$ 组。在这些组的中位数里，递归找到它们的中位数 x 。根据中位数的定义，至少有 $m/2$ 个组的中位数是 $\leq x$ 的。
- 第二步：计算“确定比 x 小”的元素 除去 x 所在的组，至少有 $m/2 - 1$ 个组，它们的中位数都小于 x 。在每一个这样的组中，由于组内 5 个元素已经排过序，中位数小于 x ，那么该组内比中位数小的 2 个元素 也一定小于 x 。加上中位数本身，每组至少有 3 个元素 小于 x 。
- 第三步：求和
 - 这些组贡献了： $3 \times (\frac{m}{2} - 1)$ 个元素。
 - 将 $m \approx n/5$ 代入： $3 \times (\frac{n/5}{2} - 1) = \frac{3n}{10} - 3$ 。

- 再加上 x 所在的组里，至少还有 **2 个元素** 比 x 小。
- **最终结果**： $(\frac{3n}{10} - 3) + 2 = \frac{3n}{10} - 1$ 。

BFPTR 的时间复杂度递推公式为：

$$T(n) \leq T(\frac{n}{5}) + T(\frac{7n}{10}) + O(n)$$

- $T(n/5)$ 是找中位数的中位数。
- $T(7n/10)$ 是最坏情况下，剩下需要处理的元素（即 $n - \frac{3n}{10} = \frac{7n}{10}$ ）。
- 由于 $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$ ，根据**主定理**或递推分析，该算法可以保证在 $O(n)$ 内完成。

教材中采用

$$T(n) \leq T(\frac{n}{5}) + T(\frac{3n}{4}) + O(n)$$

因为0.7约等于0.75，3/4更好算

在 n 较小时，递归寻找“中位数的中位数”带来的开销（如函数调用栈、数组切片等）会超过直接排序。

1. **数学上的正值要求**：在证明 $T(n) \leq cn$ 时，我们需要保证 $\frac{3n}{10} - 6 > 0$ （即 $n > 20$ ）。
2. **工程经验值**：当 n 达到 75 左右时， $\frac{3n}{10}$ 带来的“理想基准值”优势才开始显著大于递归本身的损耗。
3. **效率平衡**：对于小于 75 的数组，插入排序的常数项非常小，速度极快。强行递归反而会变慢。

动态规划

矩阵连乘

1. 定义最优解

假设矩阵连乘积 $A_i A_{i+1} \dots A_j$ 的一个 **最优计算次序**（即乘法次数最少的方案）在矩阵 A_k 和 A_{k+1} 之间将矩阵链断开。

那么，这个最优方案的总代价可以表示为：

$$m[i][j] = m[i][k] + m[k+1][j] + p_{i-1}p_kp_j$$

其中：

- $m[i][k]$ 是左子链 $A_i \dots A_k$ 的计算代价。
- $m[k+1][j]$ 是右子链 $A_{k+1} \dots A_j$ 的计算代价。
- $p_{i-1}p_kp_j$ 是最后这两部分结果相乘的代价。

2. 核心证明：反证法

我们要证明： 如果原问题是最优的，那么它的子问题（左子链和右子链）也必须是最优的。

证明过程：

1. **假设子问题不是最优的：** 假设对于左子链 $A_i \dots A_k$ ，存在另一种计算次序，其代价 $m'[i][k]$ 比我们当前方案中的 $m[i][k]$ 还要小（即 $m'[i][k] < m[i][k]$ ）。

2. **进行“剪切与粘贴”：** 我们把原方案中的左子链计算部分“剪掉”，换成这个更优的方案（“粘贴”进来）。

3. 推导矛盾： 替换后的总代价 M 将变为：

$$M = m'[i][k] + m[k+1][j] + p_{i-1}p_kp_j$$

由于 $m'[i][k] < m[i][k]$ ，显然有：

$$M < m[i][j]$$

4. **结论：** 这意味着我们找到了一个比“最优解”代价更小的方案。这与我们最初假设“ $m[i][j]$ 是最优解”相 **矛盾**。

因此，原问题的最优解所包含的子问题的解也必须是最优的。

```

1  /**
2   * MatrixChain: 计算最优乘法次数
3   * dims: 维度数组 (p), 长度为 n+1
4   * n: 矩阵的个数
5   * cost: 存储最小乘法次数的表 (m)
6   * bestSplit: 存储最优切分点的表 (s)
7   */
8  void MatrixChain(int *dims, int n, int **cost, int
9      // 1. 初始化: 单个矩阵相乘次数为 0
10     for (int i = 1; i <= n; i++) {

```

```
11         cost[i][i] = 0;
12     }
13
14     // 2. len 表示当前处理的链条长度（从长度 2 到 n）
15     for (int len = 2; len <= n; len++) {
16
17         // 3. start 表示链条的起点
18         for (int start = 1; start <= n - len + 1;
19
20             // 4. end 表示链条的终点
21             int end = start + len - 1;
22
23             // 先假设在第一个可能的位置切开，算出初始
24             // 相当于在第 start 个矩阵后面切一刀
25             cost[start][end] = cost[start][start]
26                             + dims[start - 1] *
27             bestSplit[start][end] = start;
28
29             // 5. split 表示尝试不同的切分位置 k
30             for (int split = start + 1; split < en
31
32                 // 计算公式：左边代价 + 右边代价 + 合
33                 int t = cost[start][split] + cost[
34                     + dims[start - 1] * dims[s
35
36                 // 如果发现更省力的切法，就更新它
37                 if (t < cost[start][end]) {
38                     cost[start][end] = t;
39                     bestSplit[start][end] = split;
40                 }
41             }
42         }
43     }
44 }
45
46 /**
47  * Traceback: 打印具体的加括号方式
48  */
49 void Traceback(int start, int end, int **bestSplit
50     if (start == end) {
51         cout << "A" << start;
52         return;
53     }
54
55     cout << "(";
56     // 递归打印左半部分
```

```

57     Traceback(start, bestSplit[start][end], bestSp
58
59     // 递归打印右半部分
60     Traceback(bestSplit[start][end] + 1, end, best
61     cout << " ";
62 }

```

状态方程

$$m[i, j] = \begin{cases} 0 & \text{if} \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j\} & \text{if} \end{cases}$$

最长公共子序列

最优子结构证明

假设有两个序列：

- $X = \{x_1, x_2, \dots, x_m\}$
- $Y = \{y_1, y_2, \dots, y_n\}$
- 令 $Z = \{z_1, z_2, \dots, z_k\}$ 是它们的 **任意一个 LCS**。

我们将序列的“前缀”记为 $X_i = \{x_1, \dots, x_i\}$ 。

2. 定理陈述：LCS 的最优子结构

LCS 的最优子结构性质可以分三种情况讨论：

1. 情况 1：若 $x_m = y_n$ 则 $z_k = x_m = y_n$ ，且 Z_{k-1} 是 X_{m-1} 和 Y_{n-1} 的一个 LCS。
 2. 情况 2：若 $x_m \neq y_n$ 且 $z_k \neq x_m$ 则 Z 是 X_{m-1} 和 Y 的一个 LCS。
 3. 情况 3：若 $x_m \neq y_n$ 且 $z_k \neq y_n$ 则 Z 是 X 和 Y_{n-1} 的一个 LCS。
-

3. 证明过程

情况 1 的证明（末尾相等）：

- **已知**： $x_m = y_n$ 。
- **假设**： $z_k \neq x_m$ 。那么我们可以在 Z 的末尾加上 x_m ，得到一个长度为 $k + 1$ 的公共子序列。这与 Z 是长度为 k 的“最长”公共

子序列相 **矛盾**。所以必有 $z_k = x_m = y_n$ 。

- **子问题证明**：现在我们要证 z_{k-1} 是 x_{m-1} 和 y_{n-1} 的 LCS。
- 使用 **反证法**：若存在一个更长的公共子序列 w ，其长度大于 $k-1$ 。
- 那么把 x_m 加到 w 的末尾，就会得到一个长度大于 k 的 x 和 y 的公共子序列。
- 这与 z 的长度为 k 是最优解矛盾。

情况 2 和 3 的证明（末尾不等）：

以情况 2 为例（ $x_m \neq y_n$ 且 $z_k \neq x_m$ ）：

- **分析**：既然 $z_k \neq x_m$ ，说明 x 的最后一个元素 x_m 并没有参与构成当前的 LCS。
- **结论**：因此， z 必然也是 x_{m-1} 和 y 的一个公共子序列。
- **子问题证明**：若 x_{m-1} 和 y 存在一个比 z 更长的公共子序列 w （长度 $> k$ ），那么 w 显然也是 x 和 y 的公共子序列。
- 这同样与 z 是 x 和 y 的最优解（长度为 k ）矛盾。

状态转移方程

$$dp[i][j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ dp[i-1][j-1] + 1 & \text{if } X[i] = Y[j] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{if } X[i] \neq Y[j] \end{cases}$$

- **相等时** ($X[i] = Y[j]$)：说明发现了一个公共字符。当前长度 = 左上角格子的长度 + 1。
- **不相等时** ($X[i] \neq Y[j]$)：说明当前字符对不上。我们要继承之前的“遗产”，看“去掉 x 当前字符”和“去掉 y 当前字符”哪个留下的子序列更长，取最大值。

```

1  /**
2   * m: 序列 x 的长度
3   * n: 序列 y 的长度
4   * x, y: 待比较的两个字符串（通常下标从 1 开始）
5   * c: 长度表，c[i][j] 表示 LCS 的长度
6   * b: 方向表，记录回溯路径
7   */
8  int LCSLength(int m, int n, char *x, char *y, int
9               int i, j;
10

```

```

11 // 初始化: 如果其中一个序列为空, 则 LCS 长度为 0
12 for (i = 0; i <= m; i++) c[i][0] = 0; // 第一列
13 for (i = 0; i <= n; i++) c[0][i] = 0; // 第一行
14
15 for (i = 1; i <= m; i++) {
16     for (j = 1; j <= n; j++) {
17         // 情况 1: 当前字符相等, 找到一个公共字符
18         if (x[i] == y[j]) {
19             c[i][j] = c[i - 1][j - 1] + 1; //
20             b[i][j] = 1; // i
21         }
22         // 情况 2: 字符不等, 且“上方”的 LCS 更长
23         else if (c[i - 1][j] >= c[i][j - 1]) {
24             c[i][j] = c[i - 1][j]; //
25             b[i][j] = 2; // i
26         }
27         // 情况 3: 字符不等, 且“左方”的 LCS 更长
28         else {
29             c[i][j] = c[i][j - 1]; //
30             b[i][j] = 3; // i
31         }
32     }
33 }
34 return c[m][n]; // 返回最终的最长公共子序列长度
35 }
36 /**
37  * i, j: 当前回溯到的位置 (初始调用通常为 m, n)
38  * x: 序列 x
39  * b: 方向表
40  */
41 void LCS(int i, int j, char *x, int **b) {
42     // 递归边界: 如果回溯到序列开头, 结束
43     if (i == 0 || j == 0) return;
44
45     if (b[i][j] == 1) {
46         // 方向 1: 说明 x[i] 是 LCS 的一部分
47         LCS(i - 1, j - 1, x, b); // 先递归前面的部分
48         cout << x[i]; // 后序打印, 保证字
49     }
50     else if (b[i][j] == 2) {
51         // 方向 2: 来自上方, 向上移动一行继续找
52         LCS(i - 1, j, x, b);
53     }
54     else {
55         // 方向 3: 来自左方, 向左移动一列继续找
56         LCS(i, j - 1, x, b);

```

```
57     }
58 }
```

时间复杂度： $O(m \times n)$

- 双重循环填表耗时 $m \times n$ 。
- 回溯过程最多走 $m + n$ 步。

根据最长公共子序列求解最长递减子序列

将原来的数组降序排序，与原数组进行最长公共子序列算法可得。

```
sorted_arr=sorted(arr, reverse=True)
```

最大字段和

1. 定理陈述

若 $dp[i]$ 是以 a_i 结尾的所有子段中和最大的那一个，则它必然包含以 a_{i-1} 结尾的最大子段和 $dp[i-1]$ （前提是 $dp[i-1] > 0$ ）。

2. 证明过程（反证法 / 剪切粘贴法）

第一步：定义结构

以 a_i 结尾的子段只有两种可能：

1. **单独一个元素**：即 $[a_i]$ 。
2. **与前面相连**：即 $[a_{start}, \dots, a_{i-1}, a_i]$ 。此时， $dp[i] = (\text{以 } a_{i-1} \text{ 结尾的某个子段}) + a_i$ 。

第二步：假设与矛盾推导

假设第 2 种情况是我们要的最优解（即 $dp[i]$ 的和最大），但它所包含的子问题解——“以 a_{i-1} 结尾的子段”——不是最优的。

1. **假设**：存在另一个以 a_{i-1} 结尾的子段，其和为 $dp'[i-1]$ ，且 $dp'[i-1] > dp[i-1]$ 。
2. **剪切与粘贴**：我们将原方案中以 a_{i-1} 结尾的部分“剪掉”，换成这个更优的 $dp'[i-1]$ 。
3. **得到新解**：新的总和 $M = dp'[i-1] + a_i$ 。

4. **推导矛盾**：由于 $dp'[i-1] > dp[i-1]$ ，显然 $M > dp[i-1] + a_i$ 。这说明原来的 $dp[i]$ 根本不是以 a_i 结尾的最大子段和。

结论：这与前提矛盾。因此，以 a_i^{**} 结尾的最优解，必然包含了以 a_{i-1} 结尾的最优解。^{**}

3. 状态转移方程

基于以上最优子结构性质，我们可以得出最大子段和的递归式：

$$dp[i] = \max(dp[i-1] + a_i, a_i)$$

- **如果** $dp[i-1] > 0$ ^{****}：加上前面的子段会对当前更有利，所以“拖家带口”一起走。
- **如果** $dp[i-1] \leq 0$ ^{****}：前面的子段是“负资产”，不如断舍离，从 a_i 重新开始。

4. 最终答案的获取

注意， $dp[i]$ 仅仅是以 i 结尾的最优解。整个数组的最大子段和应该是所有这些 $dp[i]$ 中的最大值：

$$\text{Result} = \max_{1 \leq i \leq n} \{dp[i]\}$$

```

1  int MaxSubSum(int n, int a[]) {
2      int sum = 0, b = 0;
3      for (int i = 0; i < n; i++) {
4          if (b > 0) b += a[i]; // 如果前面的贡献是正的
5          else b = a[i];       // 否则，从当前元素重新开始
6          if (b > sum) sum = b; // 更新全局最大值
7      }
8      return sum;
9  }
10
11 /**
12  * a: 输入数组
13  * n: 数组长度
14  * bestStart: 传引用，返回起点的下标
15  * bestEnd: 传引用，返回终点的下标
16  */
17 int MaxSubSumWithIndices(int a[], int n, int &best
18     int maxSum = -100000; // 初始为一个极小值
19     int currentSum = 0;
```

```
20     int tempStart = 0;    // 临时起点的记录员
21
22     for (int i = 0; i < n; i++) {
23         // 动态规划部分
24         if (currentSum > 0) {
25             currentSum += a[i];
26         } else {
27             // 前面的贡献是负的，舍弃之，从当前位置开始
28             currentSum = a[i];
29             tempStart = i;
30         }
31
32         // 更新全局最优解
33         if (currentSum > maxSum) {
34             maxSum = currentSum;
35             bestStart = tempStart; // 此时的临时起点
36             bestEnd = i;          // 当前位置就是终点
37         }
38     }
39     return maxSum;
40 }
```

时间复杂度： $O(N)$

0/1背包

证明（反证法）：

假设我们已经找到了容量为 C 时的最优选法，其中包含了物品 i 。

那么，如果我们将物品 i 从背包中拿走，剩下的物品对于剩余容量 $C - w_i$ 来说，也必须是最优的。

- **如果不是最优的：** 意味着在容量 $C - w_i$ 下，存在另一种选法价值更高。
- **那么：** 我们把这组更好的选法放进背包，再加上物品 i ，就会得到一个比“最优解”总价值更高的方案。
- **结论：** 这产生了矛盾，因此子问题的解也必须是最优的。

0/1 背包问题最优解的递归方程式

设：

- $dp[i][j]$ 表示在前 i 个物品中进行选择，且背包容量为 j 时的最大总价值。
- W_i 表示第 i 个物品的重量（注意：若物品编号从1开始，则对应 W_i ；若代码中数组从0开始，通常对应 W_{i-1} ）。
- V_i 表示第 i 个物品的价值。

递归方程式（状态转移方程）如下：

$$dp[i][j] = \begin{cases} 0 & \text{if } j = 0 \\ dp[i-1][j] & \text{if } j < W_i \\ \max\{dp[i-1][j], dp[i-1][j - W_i] + V_i\} & \text{if } j \geq W_i \end{cases}$$

(2) 算法伪代码描述

该算法分为两步：

1. **填表**：计算最大价值。
2. **回溯**：根据表格反推选了哪些物品。

```
1  /**
2   * v: 价值数组 (values)
3   * w: 重量数组 (weights)
4   * c: 背包总容量 (capacity)
5   * n: 物品数量
6   * m: 动态规划表, m[i][j] 表示在前 i 个物品中，容量为 j 时的最大价值
7   */
8  int Knapsack(int *v, int *w, int c, int n, int **m)
9  {
10     // 1. 初始化: 只考虑最后一个物品 n (即第 n 个物品)
11     int jMax = min(w[n] - 1, c);
12     for (int j = 0; j <= jMax; j++) {
13         m[0][j] = 0; // 容量不够装第 n 个物品
14     }
15     for (int j = w[n]; j <= c; j++) {
16         m[0][j] = v[n]; // 容量够装，价值为 v[n]
17     }
18
19     // 2. 迭代计算: 处理剩余的物品 (从 1 到 n-1)
20     for (int i = 1; i <= n - 1; i++) {
21         // 情况 A: 容量 j 不足以装下当前物品 i
22         int jMax = min(w[i] - 1, c);
23         for (int j = 0; j <= jMax; j++) {
24             m[i][j] = m[i-1][j]; // 只能继承前一个状态
```

```

25         // 情况 B: 容量 j 足够装下物品 i
26         for (int j = w[i]; j <= c; j++) {
27             // 决策: 不放物品 i (m[i-1][j]) VS 放物品 i (m[i-1][j-w[i]] + v[i])
28             m[i][j] = max(m[i - 1][j], m[i - 1][j - w[i]] + v[i]);
29         }
30     }
31
32     // 注意: 根据循环逻辑, 最终最大价值存储在 m[n-1][c]
33     return m[n-1][c];
34 }
35
36 /**
37  * m: 填好的 DP 表
38  * w: 重量数组
39  * c: 背包剩余容量
40  * n: 物品数量
41  * x: 结果数组, x[i]=1 表示选中, 0 表示未选
42  */
43 void Traceback(int **m, int *w, int c, int n, int x[]) {
44     for (int i = n - 1; i >= 1; i--) {
45         // 如果当前总价值等于“不放该物品”时的价值, 说明未选
46         if (m[i][c] == m[i - 1][c]) {
47             x[i] = 0;
48         } else {
49             // 否则, 说明选了物品 i, 并减去其重量
50             x[i] = 1;
51             c -= w[i];
52         }
53     }
54     // 处理最后一个物品 (在 DP 表中对应的第 0 行)
55     x[0] = (m[0][c] > 0) ? 1 : 0;
56 }

```

(3) 算法时间复杂度分析

对上述算法进行分析:

1. 填表过程 :

- 算法使用了两层嵌套循环。
- 外层循环遍历物品数量 i , 从 1 到 n , 执行 n 次。
- 内层循环遍历背包容量 j , 从 1 到 C , 执行 C 次。

- 循环体内部的操作（比较、加法、赋值）都是常数时间操作 $O(1)$ 。
- 因此，填表的时间复杂度为 $O(n \times C)$ 。

1. 回溯过程：

- 循环从 n 递减到 1 ，执行 n 次。
- 内部操作为常数时间。
- 这部分的时间复杂度为 $O(n)$ 。

总时间复杂度：

$$T(n) = O(n \times C) + O(n) \approx O(n \times C)$$

注意：

这里的 C 是背包的容量数值，而不是输入的规模。如果 C 非常大（例如 $C = 2^n$ ），该算法的效率会变得很低。因此，严谨地说，这是一个**伪多项式时间（Pseudo-polynomial time）**算法。

贪心算法

活动安排

算法伪代码及复杂度分析

核心思路：

为了安排尽可能多的活动，我们应该总是选择结束时间最早且与当前已选活动不冲突的活动。这样可以为剩下的时间段留出最大的空间，从而容纳更多的活动。

伪代码

```
1  /**
2   * 贪心算法求解活动安排问题
3   * * 参数说明：
4   * n    ：活动的总数量
5   * s[]  ：活动开始时间数组（下标从1开始，s[1]...s[n]）
6   * f[]  ：活动结束时间数组（下标从1开始，f[1]...f[n]）
7   * ***重要前提***：f[] 必须已经按照非递减顺序（从小到大
```

```
8   * A[] : 布尔数组, 用于记录结果。A[i] = true 表示选中
9   */
10 void GreedySelector(int n, Type s[], Type f[], bool A[])
11 {
12     // 步骤 1: 贪心选择策略 - 总是先选择结束时间最早的
13     // 因为 f[] 已排序, 所以 f[1] 是结束最早的
14     A[1] = true;
15
16     int j = 1; // j 用于记录“上一个被选中的活动”的索引
17
18     // 步骤 2: 依次检查剩余的 n-1 个活动
19     for (int i = 2; i <= n; i++) {
20         // 检查相容性:
21         // 如果当前活动 i 的开始时间 (s[i]) 晚于或等于
22         // 上一个被选活动 j 的结束时间 (f[j])
23         if (s[i] >= f[j]) {
24             A[i] = true; // 标记选中
25             j = i;       // 更新 j, 将当前活动 i 设为
26                             // 上一个被选活动
27         } else {
28             A[i] = false; // 时间冲突, 不选中
29         }
30     }
31 }
```

对该函数的复杂度分析如下:

- **循环次数**: 代码中只有一个 for 循环, 循环变量 i 从 2 遍历到 n 。这意味着循环体总共执行了 $n - 1$ 次。
- **循环内部操作**: 在循环体内, 执行的是简单的比较 ($>=$) 和赋值操作。这些都是常数时间操作, 记为 $O(1)$ 。
- **总运算量**: $(n - 1) \times O(1)$ 。

结论:

该函数本身的时间复杂度为 $O(n)$ (线性时间复杂度)。

说明算法采用的贪心策略

该算法采用的贪心策略是: **最早结束时间优先 (Earliest Finish Time First)**。

解释:

在所有与当前已选活动相容（不冲突）的剩余活动中，总是选择结束时间最早的那个活动。

- **理由：** 结束时间越早，剩余的可安排时间资源就越多，从而更有可能容纳后续的活动。这是一种“目光短浅”但在此问题中能导致全局最优的策略。

贪心选择性质的证明（以“替换法”证明）

这一步的目的是证明：**全局最优解中，一定可以包含贪心选择出来的那个局部最优项。**

- **设定前提：** 设 $E = \{1, 2, \dots, n\}$ 是按结束时间 f_i 非减序排列的活动集合，其中活动 1 具有最早结束时间。
- **假设最优解：** 设 $A \subseteq E$ 是该问题的一个最优解，且 A 中的一个活动是 k 。
- **分类讨论：**
 1. **如果 $k = 1$ ：** 说明最优解 A 已经包含了贪心选择的活动 1，证明结束。
 2. **如果 $k > 1$ ：** 说明最优解没有选活动 1，而是选了 k 。此时我们构造一个新的集合 $B = (A - \{k\}) \cup \{1\}$ ，即 **用活动 1 替换掉活动 k** 。
- **证明可行性：** 因为活动 1 的结束时间 $f_1 \leq f_k$ （因为集合是按结束时间排序的），而 k 与 A 中后续活动是不冲突的，那么结束得更早的活动 1 自然也不会与后续活动冲突。
- **结论：** 由于 B 中的活动数量和 A 一样多，且 B 也是相容的，所以 B 也是一个最优解。这证明了 **总存在一个包含贪心选择（活动 1）的最优解。**

最优子结构性质（以“反证法”证明）

这一步的目的是证明：**做出贪心选择后，原问题的最优解包含了子问题的最优解。**

简化问题： 当我们选择了活动 1 后，剩下的可选活动集合变为 $E' = \{i \in E : s_i \geq f_1\}$ （即所有开始时间晚于活动 1 结束时间的活动）。**反**

证逻辑：

- 设 A 是原问题 E 的最优解，则 $A' = A - \{1\}$ 应该是子问题 E' 的最优解。
- 假设 A'^{**} 不是 E' 的最优解^{**}：那么一定存在另一个解 B' ，它包含的活动数量比 A' 更多。
- 推导矛盾：如果我们把活动 1 加回到 B' 中，得到 $B = B' \cup \{1\}$ 。由于 B' 活动更多，那么 B 的活动数量就会比原始最优解 A 还要多。
- 结论：这与“ A 是最优解”的前提矛盾。因此，每一步贪心选择所留下的子问题，其最优解与贪心选择合并后，一定是原问题的最优解。

单源最短路径

形式化描述

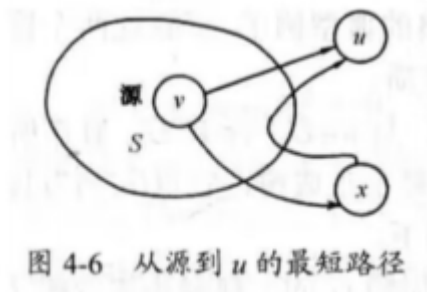
对图 $G(V, E)$, 源点 v , 从三方面描述问题

- 源点 v , 图中顶点集合 V , 各顶点 $u \in V$ 的全局最短路径及其长度 $d(v, u)$
 - 算法迭代过程中保持不变
- 用于构造最短路径的当前顶点集合 S_i ,
 - 不断增加，定义了不同规模的子问题
- 指标：相对于现有 S_i , 对各顶点 $u \in V$, $\text{dist}[u]$
 - 随着 S_i 的变化， $\text{dist}[u]$ 可能需要不断更新

问题/子问题描述： $[S_i, \{ \langle u, \text{dist}[u] \rangle \}]$, $S_i \subseteq V$, $u \in V$

- 最简子问题： $[\{u\}, \{ \langle u, 0 \rangle \}]$, S_i 只包含源点 u
- 原问题： $[V, \{ \langle u, \text{dist}[u] \rangle \}] = [V, \{ \langle u, d(u, v) \rangle \}]$, $S_i = V$

贪心选择性质的证明 (图 4-6)

图 4-6 从源到 u 的最短路径

1766578742560

这一部分是为了证明：**为什么每次从“未确定集合”中挑选 dist^{**} 值最小的顶点 u ，这个 $\text{dist}[u]$ 就一定是源点到 u 的最短路径距离？****

证明思路：反证法

1. **假设：** 存在一条从源点 v 到 u 的路径，其长度比当前的 $\text{dist}[u]$ 还要短。
2. **寻找矛盾点：**
 - 如图 4-6 所示，这条假设的“更短路径”必然会离开已确定集合 S 。
 - 设 x 是这条路径上离开 S 后的**第一个**顶点。
 - 那么，从源点 v 到 x 的路径长度为 $d(v, x)$ 。因为 x 在 S 之外，且它是路径上的一个中间点，根据边权非负的原则，到达 x 的距离 $\text{dist}[x]$ 必然满足： $\text{dist}[x] \leq d(v, x) \leq$ 假设的 u 的路径长度 $< \text{dist}[u]$
3. **结论：** * 如果 $\text{dist}[x] < \text{dist}[u]$ ，那么按照 Dijkstra 算法的贪心策略，算法应该先选择顶点 x 加入 S ，而不是先选择 u 。
 - 这与“算法当前选择了 u ”相矛盾。
 - 因此，不存在更短的路径， $\text{dist}[u]$ 就是最短路径长度。

最优子结构性质的证明 (图 4-7)

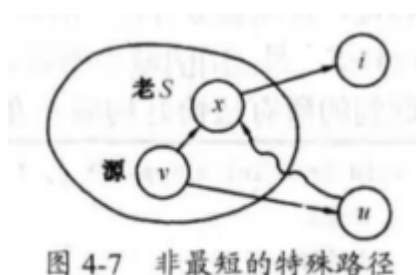


图 4-7 非最短的特殊路径

1766578757348

这一部分是为了证明：**当我们将顶点 u^{**} 加入集合 S 后，更新其他顶点 i 的 $dist$ 值时，为什么只需要考虑经过 u 的直接边？****

证明逻辑：

当 u 加入 S 后，可能会产生新的通往顶点 i 的特殊路径（特殊路径指：除了终点 i 外，其他点都在 S 中的路径）。

1. **直接路径**：路径为 $v \rightarrow \dots \rightarrow u \rightarrow i$ 。其长度为 $dist[u] + c[u]$ $[i]$ 。如果这个值比原来的 $dist[i]$ 小，我们就更新它。
2. **间接路径 (图 4-7)**：如果新路径是经过 u 后，又回到了 S 中的某个老顶点 x ，最后再到达 i 。
 - **分析**：因为顶点 x 是“老 S ”中的成员，它比 u 更早加入 S 。
 - 根据算法逻辑，源点到 x 的最短距离在 u 加入之前就已经确定了。
 - **结论**：经过 u 再绕回 x 的路径长度，必然大于源点直接到 x 的长度。因此，这种“绕路”的情况不需要考虑。

总结：每次加入新点 u 后，只需要通过 u 对其相邻的、不在 S 中的顶点进行一次“松弛操作”（Relaxation）即可。

代码实现

1. 选离原点最近且未访问的点
2. 标记该点访问过
3. 更新所有未访问结点到原点距离

```

1
2  /*
3   * Dijkstra 核心算法
4   * 参数：
5   * n: 顶点数
6   * v: 源点
7   * dist[]: 距离数组 (输出)
8   * prev[]: 前驱数组 (输出)
9   * c[][]: 邻接矩阵 (输入)
10  * m是整形最大值
11  */
12
13 void Dijkstra(int n, int v, int dist[], int prev[])
14 {
15     bool s[n + 1]; // 集合 S, s[i]=true 表示点 i 的

```

```
16
17 // --- 步骤 1: 初始化 (Initialization) ---
18 for (int i = 1; i <= n; i++)
19 {
20     dist[i] = c[v][i]; // 初始距离为源点直连的距
21     s[i] = false;      // 所有点初始都不在集合 S
22
23     // 设置前驱节点
24     if (dist[i] == M) {
25         prev[i] = 0; // 如果不可达, 前驱置空
26     }
27     else {
28         prev[i] = v; // 如果可达, 前驱就是源点 v
29     }
30 }
31
32 dist[v] = 0; // 源点到自己的距离为 0
33 s[v] = true; // 将源点加入集合 S
34
35 // --- 步骤 2: 循环 n-1 次, 确定剩余 n-1 个点的最
36 for (int i = 1; i < n; i++)
37 {
38     int temp = M;
39     int u = v;
40
41     // A. 贪心选择 (Selection)
42     // 在未被访问的点 (V-S) 中, 寻找距离源点最近的点
43     for (int j = 1; j <= n; j++)
44     {
45         if ((!s[j]) && (dist[j] < temp))
46         {
47             u = j;
48             temp = dist[j];
49         }
50     }
51
52     // 如果找不到更近的点 (说明剩下的点都不可达),
53     if (temp == M) break;
54
55     s[u] = true; // 将点 u 加入集合 S, 表示它的最短距离已确定
56
57     // B. 松弛更新 (Relaxation)
58     // 借道 u, 看能否缩短源点到其他未访问点 j 的距离
59     for (int j = 1; j <= n; j++)
60     {
61         // 条件: j 不在 S 中, 且 u 到 j 有边
```

```

62         if ((!s[j]) && (c[u][j] < M))
63         {
64             int newdist = dist[u] + c[u][j]; /
65
66             // 如果新距离比原来的 dist[j] 更短,
67             if (newdist < dist[j])
68             {
69                 dist[j] = newdist;
70                 prev[j] = u; // 关键: 记录 j 的
71             }
72         }
73     }
74 }
75 }
76
77 /*
78  * 递归输出路径
79  * 原理: 利用 prev 数组不断找前驱, 直到回溯到源点, 利用
80  */
81 void Traceback(int v, int i, int prev[])
82 {
83     // 递归终止条件: 找到了源点
84     if (v == i)
85     {
86         cout << i; // 打印原点
87         return;
88     }
89
90     // 递归调用: 先打印前面的路径
91     Traceback(v, prev[i], prev);
92
93     // 再打印当前点
94     cout << "->" << i;
95 }

```

$$T(n) = O(n) + (n - 1) \times [O(n) + O(n)] = O(n^2)$$

- 最终复杂度: $O(n^2)$
- 空间复杂度: $O(n^2)$ (因为使用了邻接矩阵 $c[n+1][n+1]$)

分数背包问题

贪心选择性质证明 (证明“第一步”拿单位价值最高的是对的)

我们使用 **替换法 (Substitution)** 来证明，这种方法在你提供的最优装载问题证明中也有体现。

- **设定前提**：设物品已按单位价值从大到小排序，即

$$\frac{v_1}{w_1} \geq \frac{v_2}{w_2} \geq \dots \geq \frac{v_n}{w_n}。$$

- **假设最优解**：设 $A = \{x_1, x_2, \dots, x_n\}$ 是一个最优解，其中 x_i 是物品 i 装入的比例 ($0 \leq x_i \leq 1$)。
- **比较贪心解**：贪心算法会尽可能多地装入物品 1。如果 A 中 $x_1 < 1$ 且背包还有空间，或者 x_1 不是在当前限制下的最大比例，那么 A 就没有完全遵循贪心策略。

- **构造新解**：

1. 如果在 A 中物品 1 没有装满，我们可以从 A 中剔除一部分单位价值较低的物品 j (其中 $j > 1$)，换成相同重量的物品 1。
2. 设换入的重量为 Δw 。

- **证明价值增加**：

- 原价值减少了 $\Delta w \cdot \frac{v_j}{w_j}$ ，新价值增加了 $\Delta w \cdot \frac{v_1}{w_1}$ 。
- 因为 $\frac{v_1}{w_1} \geq \frac{v_j}{w_j}$ ，所以总价值 $\sum v_i x_i$ 必然不会减少。
- **结论**：这说明总存在一个包含“尽可能多地装入单位价值最高物品”的最优解。

最优子结构性证明 (证明“剩下的”部分依然是最优的)

这一步利用 **反证法**，证明原问题的最优解包含子问题的最优解，逻辑与你图片中的 Step 2 一致。

- **简化问题**：假设我们已经根据贪心选择，装入了重量为 w 的物品 1。此时，剩余的背包容量为 $C' = C - w$ ，剩余物品集合为 $E' = \{2, 3, \dots, n\}$ 。
- **反证逻辑**：
 - 设 S 是原问题的最优解。那么 S 去掉物品 1 后的部分 S' ，必须是子问题 (容量 C' ，集合 E') 的最优解。
 - **假设 S'^* 不是子问题的最优解****：意味着存在另一个方案 S'' ，在容量 C' 下能获得比 S' 更高的价值。
 - **推导矛盾**：如果我们把物品 1 加回到 S'' 中，得到的新解总价值将超过 S 。这与“ S 是原问题的最优解”矛盾。

- **结论**：每一步贪心选择后，剩下的问题依然满足最优子结构，可以递归地使用贪心策略。

贪心解决的是分数背包，对于0/1背包不行

```
1 void Knapsack(int n, float M, float v[], float w[])
2     // 关键步骤：必须按照“单位价值” (v/w) 从大到小排序
3     // 注意：这里的 v[] 和 w[] 必须同步交换顺序
4     Sort(n, v, w);
5
6     int i;
7     // 初始化：将所有物品的选择比例置为 0
8     for (i = 1; i <= n; i++) x[i] = 0;
9
10    float c = M; // c 代表当前背包的剩余容量 (Current Capacity)
11
12    // 循环：依次考察排序后的物品
13    for (i = 1; i <= n; i++) {
14        // 如果当前物品 i 的重量大于剩余容量 c，说明装
15        // 不下，跳出循环
16        if (w[i] > c) break;
17
18        // 否则，将物品 i 全部装入
19        x[i] = 1;
20
21        // 更新剩余容量
22        c -= w[i];
23    }
24
25    // 处理“跳出循环”时的那个物品（即只能装一部分的物品）
26    // 如果 i <= n，说明是 break 出来的，或者是刚好处
27    // 理完所有物品
28    if (i <= n)
29        x[i] = c / w[i]; // 装入比例 = 剩余容量 / 物品重量
30 }
```

时间复杂度分析

1. 排序 (Sort)：这是算法中最耗时的部分。使用高效排序（如快速排、归并），时间复杂度为 $O(n \log n)$ 。
2. 循环 (for)：代码中只有一个从 1 到 n 的循环，最坏情况遍历所有物品，时间复杂度为 $O(n)$ 。

总时间复杂度：

$$T(n) = O(n \log n) + O(n) = O(n \log n)$$

最优装载问题

最优装载问题 (Optimal Loading Problem) 是贪心算法中最经典、最直观的案例之一。虽然它看起来像背包问题，但它的目标略有不同，因此解法也更加简单。

以下是关于该问题的详细描述和贪心策略。

问题描述 (Problem Description)

1. 场景设定

假设有一艘轮船（或者一辆卡车），它的最大载重量为 C 。

码头上有 n 个集装箱（古董、货物），第 i 个集装箱的重量为 w_i 。

注意：与背包问题不同的是，这些集装箱 **没有“价值”的区别**（或者说每个集装箱的价值都视为 1）。

2. 目标

我们的目标是在不超过轮船载重量 C 的前提下，**装入尽可能多的集装箱数量**。

3. 数学形式化

设 x_i 为变量，当 $x_i = 1$ 时表示装入第 i 个集装箱，当 $x_i = 0$ 时表示不装入。

- 目标函数：最大化装入的数量

$$\max \sum_{i=1}^n x_i$$

- 约束条件：

$$\sum_{i=1}^n w_i x_i \leq C, \quad x_i \in \{0, 1\}$$

贪心策略 (Greedy Strategy)

对于这个问题，直觉往往就是正确的。

1. 核心策略：重量最轻者优先 (Lightest First)

为了装入更多数量的集装箱，我们应该尽可能留出空间。每一个被选中的集装箱都应该尽可能少地消耗船的载重。因此，贪心策略就是：**优先选择重量最小的物品。**

2. 算法步骤

1. **排序**：将所有集装箱的重量 w_i 按**从小到大（非递减）**的顺序排序。
2. **循环选择**：
 - 从最轻的集装箱开始，依次尝试装入。
 - 如果当前集装箱的重量 $\leq$ 船的剩余载重，就装入，并减少剩余载重。
 - 如果当前集装箱装不下，由于后面的更重，肯定也装不下，算法直接结束。

证明贪心选择策略

这一步的核心是证明：在所有最优方案中，一定存在一个方案包含了重量最轻的 1 号集装箱。

- **前提条件**：假设集装箱已按重量从小到大排序 $w_1 \leq w_2 \leq \dots \leq w_n$ 。
- **假设最优解**：设 $X = \{x_1, x_2, \dots, x_n\}$ 是一个最优解（装入箱子最多的方案），其中 k 是该方案中装入的最轻集装箱的索引，即 $k = \min \{i | x_i = 1\}$ 。
- **分类讨论**：
 1. **情况 1** ($k = 1$)：如果最优解本来就包含了最轻的 1 号箱，证明结束。
 2. **情况 2** ($k > 1$)：如果最优解没装 1 号箱，而是装了更重的 k 号箱。
- **构造新解**：我们用 1 号箱替换掉 k 号箱，得到新方案 Y 。即令 $y_1 = 1, y_k = 0$ ，其余物品选取状态与 X 保持不变。
- **证明新解的合法性与最优性**：

- **重量限制**：由于 $w_1 \leq w_k$ ，替换后的总重量 $\sum w_i y_i = w_1 - w_k + \sum w_i x_i \leq \sum w_i x_i \leq C$ 。这意味着新方案 Y 依然没有超过载重。
- **箱子数量**：新方案 Y 的箱子总数 $\sum y_i$ 等于原最优解 X 的箱子总数 $\sum x_i$ 。
- **结论**：既然 Y 合法且数量也是最多的，说明 **优先装入最轻的集装箱可以使总箱数最大化**。

2. 最优子结构性性质证明（证明“剩下的”也能这么选）

这一步证明：当你装入最轻的集装箱后，剩下的问题依然可以用同样的贪心策略解决。

- **原问题与子问题**：设原问题是在容量为 C 时从 $\{1, 2, \dots, n\}$ 中选。当你根据贪心策略选了 1 号箱 ($x_1 = 1$) 后，问题变成了：在容量为 $C - w_1$ 的情况下，从剩余的集装箱 $\{2, 3, \dots, n\}$ 中进行选择。
- **证明逻辑**：如果 $\{x_1, x_2, \dots, x_n\}$ 是原问题的最优解，那么其中的子集 $\{x_2, x_3, \dots, x_n\}$ 必然是容量为 $C - w_1$ 时子问题的最优解。
- **直观理解**：如果子问题有更好的解（能装更多箱子），那么把它和 1 号箱结合，就会得到一个比原最优解还要好的方案，这与前提矛盾。

代码

```
1 void Loading(int x[], Type w[], Type c, int n) {
2     // 创建一个辅助数组 t，大小为 n+1（适应 1-based i
3     int *t = new int[n+1];
4
5     // 关键步骤：间接排序
6     // 输入：w（重量），n（个数）
7     // 输出：t（排序后的索引）
8     // 此时，t[1] 是最轻物品的ID，t[n] 是最重物品的ID
9     Sort(w, t, n);
10
11     // 初始化解向量 x，默认全部为 0（都不装）
12     for (int i = 1; i <= n; i++)
13         x[i] = 0;
14
15     // 贪心循环
16     // 条件1: i <= n（还没遍历完）
```

```

17      // 条件2: w[t[i]] <= c (当前第 i 轻的物品重量 <=
18      for (int i = 1; i <= n && w[t[i]] <= c; i++)
19          // x[t[i]] = 1: 标记原始编号为 t[i] 的物品被
20          x[t[i]] = 1;
21
22      // 更新剩余容量: 减去当前第 i 轻物品的重量
23      c -= w[t[i]];
24  }
25
26      // 注意: 这里缺少了 delete[] t; 存在内存泄漏风险
27  }

```

总时间复杂度 $T(n)$ 是各部分之和:

$$T(n) = \underbrace{O(n \log n)}_{\text{排序}} + \underbrace{O(n)}_{\text{初始化}} + \underbrace{O(n)}_{\text{贪心选择}}$$

最终时间复杂度:

$O(n \log n)$

哈夫曼编码

贪心选择性质

命题: 设字符集 C 中频率最低的两个字符为 x 和 y , 则存在一棵最优前缀码树 T , 使得 x 和 y 是深度最深的兄弟叶节点。

证明过程 (交换论证法):

1. **初始假设:** 设 T 是一棵最优前缀码树。在 T 中, 设 b 和 c 是深度最深 (码长最长) 的一对兄弟叶节点。不妨设 $f(b) \leq f(c)$ 且已知在 T 中有两个非最深的频率最高的节点, x, y , 不妨设 $f(x) \leq f(y)$ 。
2. **构造新树 T^{****} :** 由于 x, y 是全局频率最小的, 必有 $f(x) \leq f(b)$ 且 $f(y) \leq f(c)$ 。我们将 x 与 b 交换, y 与 c 交换, 得到新树 T' 。
3. **计算代价差:** 对比 T 与 T' 的总权路径长度 (代价):

$$B(T) - B(T') = \sum_{c \in C} cf(c)d_T(c) - \sum_{c \in C} cf(c)d_{T'}(c)$$

$$B(T) - B(T') = [f(x)d_T(x) + f(b)d_T(b) + f(y)d_T(y) + f(c)d_T(c)] - [f(x)d_{T'}(x) + f(b)d_{T'}(b) + f(y)d_{T'}(y) + f(c)d_{T'}(c)]$$

$$B(T) - B(T') = f(x)d_T(x) + f(b)d_T(b) + f(y)d_T(y) + f(c)d_T(c) - [f(x)d_{T'}(b) + f(b)d_{T'}(x) + f(y)d_{T'}(c) + f(c)d_{T'}(y)]$$

经过代数化简（仅 x, y, b, c 的项未抵消）：

$$B(T) - B(T') = (f(b) - f(x))(d_T(b) - d_T(x)) + (f(c) - f(y))(d_T(c) - d_T(y))$$

4. 结论：

- 因为 $f(b) \geq f(x)$ 且 $f(c) \geq f(y)$ （频率最小性）
- 因为 $d_T(b) \geq d_T(x)$ 且 $d_T(c) \geq d_T(y)$ （ b, c 在最深层）
- 所得结果 ≥ 0 ，即 $B(T') \leq B(T)$ 。
- 由于 T 已经是最优的， T' 的代价只能等于 T 。故 T' 的码长仍然是最短的，即 T' 是最优前缀码，并且其最小频率的 x, y 具有最深的深度（最长的编码），且只有最后一位不同

这证明了：**贪心选择（合并最小的两个）不会错过全局最优解。**

最优子结构性性质

命题：将 x 和 y 合并为新字符 z （频率 $f(z) = f(x) + f(y)$ ），得到子问题 C' 。若 T' 是 C' 的最优树，则将 z 替换回 x, y 得到的树 T 必是原问题 C 的最优树。

关键点 1：建立原问题与子问题的代价关系

- 递推表达式：原树 T 的平均码长 $B(T)$ 可以用子树 T' 的码长 $B(T')$ 表示为： $B(T) = B(T') + f(x) + f(y)$
- **深度关系推导：**因为 x, y 是 z 的子节点，所以深度满足 $d_T(x) = d_T(y) = d_{T'}(z) + 1$ 。
- 代数关系： $f(x)d_T(x) + f(y)d_T(y) = (f(x) + f(y))(d_{T'}(z) + 1) = f(x) + f(y) + (f(x) + f(y))d_{T'}(z) = f(x) + f(y) + f(z)d_{T'}(z)$

关键点 2：采用反证法证明 T' 是最优的

1. **假设：**存在另一个树 T'' 是子问题 C' 的更优前缀码，即满足 $B(T') > B(T'')$ 。

2. **构造**：在树 T'' 中，将节点 z （作为叶子节点）替换为其子节点 x, y ，从而得到原问题 C 的一个解 T''' 。

3. 计算 T''' 的代价：根据上述关键点 1 的关系，可得：

$$B(T''') = B(T'') + f(x) + f(y)$$

4. 得出矛盾：由于假设 $B(T'') < B(T')$ ，则：

$$B(T''') = B(T'') + f(x) + f(y) < B(T') + f(x) + f(y) = B(T)$$

这意味着 T''' 是比最优树 T 更优的解，这与“ T 是原问题的最优解”这一前提矛盾。

结论：因此 T' 必然是子问题的最优解，最优子结构性质得证。

代码实现

```

1
2
3 // 哈夫曼树节点结构
4 struct Node {
5     char ch;          // 字符（内部节点可为空）
6     int freq;         // 频率
7     Node *left, *right;
8
9     Node(char c, int f) : ch(c), freq(f), left(nullptr), right(nullptr) {}
10 };
11
12 // 比较仿函数：构造最小堆（Min-Heap）
13 struct Compare {
14     bool operator()(Node* l, Node* r) {
15         return l->freq > r->freq; // 频率小的优先级高
16     }
17 };
18
19 // 递归遍历树，生成编码表
20 void generateCodes(Node* root, string str, map<char, string>& huffmanCode) {
21     if (!root) return;
22
23     // 叶子节点：存储编码
24     if (!root->left && !root->right) {
25         huffmanCode[root->ch] = str;
26     }
27
28     generateCodes(root->left, str + "0", huffmanCode);
29     generateCodes(root->right, str + "1", huffmanCode);

```

```
30 }
31
32 // 构建哈夫曼树并返回根节点
33 Node* buildHuffmanTree(const map<char, int>& freqs
34     priority_queue<Node*, vector<Node*>, Compare>
35
36     // 1. 将所有字符作为叶子节点放入优先队列
37     for (auto const& [ch, freq] : freqs) {
38         pq.push(new Node(ch, freq));
39     }
40
41     // 2. 贪心合并：不断取出两个最小频率节点
42     while (pq.size() != 1) {
43         Node *left = pq.top(); pq.pop();
44         Node *right = pq.top(); pq.pop();
45
46         // 创建新内部节点 z，频率为子节点之和  $f(z) =$ 
47         int sum = left->freq + right->freq;
48         Node *node = new Node('\0', sum);
49         node->left = left;
50         node->right = right;
51         pq.push(node);
52     }
53
54     return pq.top(); // 最终剩下的即为根节点
55 }
```

时间复杂度分析

- 构建堆： $O(n)$ ，其中 n 是字符集大小。
- 合并过程：共进行 $n - 1$ 次合并，每次弹出和压入堆的操作为 $O(\log n)$ ，总计 $O(n \log n)$ 。
- 总复杂度： $O(n \log n)$ 。

最小生成树

MST 性质 (Cut Property)

设 $G = (V, E)$ 是一个连通带权无向图，边 (u, v) 的权值为 $w(u, v)$ 。

性质描述： 设 U 是顶点集 V 的一个真子集（即 $U \subset V$ 且 $U \neq \emptyset$ ）。如果 (u, v) 是连接 U 和 $V - U$ 的所有边（称为**横切边**）中权值最小的一条边，那么一定存在一棵最小生成树 (MST) 包含边 (u, v) 。

证明 (反证法)

1. **假设**: 假设图 G 的任何一棵最小生成树都不包含这条权值最小的横切边 (u, v) 。

2. **构造矛盾**:

- 设 T 是 G 的一棵最小生成树。根据假设, $(u, v) \notin T$ 。
- 由于 T 是生成树, 它包含连接所有顶点的路径。因此, 在 T 中必然存在一条从 u 到 v 的唯一路径。
- 因为 $u \in U$ 且 $v \in V - U$, 这条路径必然要从集合 U 跨越到集合 $V - U$ 。这意味着路径上至少有一条边 (x, y) 也是横切边 (即 $x \in U, y \in V - U$)。
- 根据 MST 性质的前提条件, (u, v) 是所有横切边中权值最小的, 所以有 $w(u, v) \leq w(x, y)$ 。
- 现在, 我们构造一个新的图 T' : 将边 (u, v) 加入 T , 并删除边 (x, y) 。

$$T' = (T - \{(x, y)\}) \cup \{(u, v)\}$$

- **连通性分析**: 加入 (u, v) 会在 T 中形成一个环, 而删除环上的另一条边 (x, y) 会打破这个环, 同时保持图的连通性 (因为 (x, y) 和 (u, v) 都在跨越切口的路径上)。因此 T' 仍然是一棵生成树。

◦ **权值分析**:

$$w(T') = w(T) - w(x, y) + w(u, v)$$

由于 $w(u, v) \leq w(x, y)$, 则 $w(T') \leq w(T)$ 。

3. **结论**:

- 如果 $w(u, v) < w(x, y)$, 则 $w(T') < w(T)$, 这与 T 是最小生成树矛盾。
- 如果 $w(u, v) = w(x, y)$, 则 $w(T') = w(T)$, 说明 T' 也是一棵最小生成树, 且它包含了 (u, v) 。这与“任何 MST 都不包含 (u, v) ”的假设矛盾。

综上所述, 假设不成立, 边 (u, v) 一定属于某棵最小生成树。

Prim 算法

Prim 算法是 MST 性质的直接应用。

算法思路:

1. 维护一个集合 S ，初始时只包含任意一个起始顶点（例如 $S = \{1\}$ ）。
2. 在所有连接 S 和 $V - S$ 的边中，选择权值最小的那条边 (u, v) （其中 $u \in S, v \in V - S$ ）。
3. 将顶点 v 加入集合 S ，并将边 (u, v) 加入最小生成树。
4. 重复步骤 2 和 3，直到 S 包含所有顶点。

时间复杂度:

- 使用邻接矩阵 + 数组实现: $O(V^2)$

```

1  /**
2   * 图的结构体定义
3   */
4  typedef struct {
5      char vexs[MAX];          // 顶点集合 (例如: 'A', 'B', ...)
6      int matrix[MAX][MAX];    // 邻接矩阵 (存储边权值)
7      int vexnum;              // 顶点数量
8      int edgenum;             // 边数量
9  } Graph;
10
11
12 /**
13  * Prim 最小生成树算法
14  * 核心思想: 贪心算法。每次从“未被选入生成树的顶点”中，
15  *
16  * @param G      图结构体 (包含 vexs 顶点数组, matrix 邻接矩阵)
17  * @param start  从第 start 个顶点开始生成 (索引)
18  */
19 void prim(Graph G, int start) {
20
21     // weights[] 是核心辅助数组。
22     // 含义: weights[i] 保存了顶点 i 到“当前最小生成树”的最小边权值。
23     // 特殊约定: 如果 weights[i] == 0, 表示顶点 i 已加入生成树。
24     int weights[G.vexnum];
25
26     int index = 0;          // 结果数组 prim 的索引
27     char prim[G.vexnum];    // 用于保存最小生成树的顶点
28     int total = 0;          // 记录最小生成树的总权值
29
30     // --- 步骤 1: 初始化 ---
31

```

```
32 //先把起始顶点 start 加入集合，所以距离置为 0
33 weights[start] = 0;
34
35 // 初始化 weights 数组
36 // 将 start 顶点到其他所有顶点的距离存入 weights
37 for(int i = 0; i < G.vexnum; i++) {
38     weights[i] = G.matrix[start][i];
39 }
40
41 // 将起始顶点放入结果数组
42 prim[index++] = G.vexs[start];
43
44 // --- 步骤 2: 循环寻找剩余的 n-1 个顶点 ---
45
46 for(int i = 0; i < G.vexnum; i++) {
47     // 由于 start 已经加入，跳过一次循环计数（这里
48     if(i == start) continue;
49
50     int k = -1; // 用于记录当前找到的距离最
51     int min = INF; // min 用来存放当前的最小权
52
53     // 2.1 寻找最近顶点:
54     // 遍历所有顶点，找出未加入生成树 (weights!=0
55     for(int j = 0; j < G.vexnum; j++) {
56         // weights[j] != 0 代表该点还没被选中
57         // weights[j] < min 代表发现了更近的点
58         if(weights[j] != 0 && weights[j] < min
59             min = weights[j];
60             k = j; // 记录下这个顶点的下标
61         }
62     }
63
64     // 如果 k 还是 -1，说明剩下的顶点都无法到达（
65     if(k == -1) break;
66
67     // 2.2 将找到的顶点 k 加入生成树:
68     weights[k] = 0; // 标记为 0，表
69     prim[index++] = G.vexs[k]; // 存入结果数组
70     total += min; // 累加权值
71
72     // 2.3 更新距离（松弛操作）:
73     // 新加入顶点 k 后，可能通过 k 到达其他顶点的
74     // 所以要遍历 k 的所有邻接点，更新 weights 数
75     for(int j = 0; j < G.vexnum; j++) {
76         // 如果顶点 j 还没加入生成树 (weights[j]
77         // 且从 k 到 j 的距离 (G.matrix[k][j])
```

```
78         if(weights[j] != 0 && weights[j] > G.m
79             weights[j] = G.matrix[k][j]; // 更
80     }
81 }
82 }
83 }
84
85 // --- 第三步：打印结果 ---
86     printf("PRIM最小生成树结果 (起始点 %c):\n", G.v
87
88     // 打印节点顺序
89     printf("节点生成顺序: ");
90     for (int i = 0; i < index; i++) {
91         printf("%c ", prim[i]);
92     }
93     printf("\n");
94
95     // 打印总权值
96     printf("最小生成树总权值: %d\n", total);
97
98 }
```

Kruskal 算法

Kruskal 算法也是基于 MST 性质，但它是从边的角度出发。

算法思路：

1. 将图中所有边按权值从小到大排序。
2. 初始时，每个顶点自成一个连通分量。
3. 依次考察排序后的每条边 (u, v) :
 - 如果 u 和 v 属于不同的连通分量，则选择这条边，并合并这两个分量（因为这条边是连接这两个分量的最短边，符合 MST 性质）。
 - 如果 u 和 v 已经属于同一个连通分量，则舍弃这条边（否则会形成环）。
4. 重复步骤 3，直到选择了 $n - 1$ 条边。

实现细节： 通常使用 **并查集 (Union-Find)** 数据结构来快速判断两个顶点是否属于同一个连通分量以及合并分量。

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define MAX 100          // 最大顶点数
5  #define INF 65535        // 无穷大
6
7  /**
8   * 边结构体：用于 Kruskal 算法中的边排序
9   */
10 typedef struct {
11     char start;    // 起点 (例如 'A')
12     char end;      // 终点 (例如 'B')
13     int weight;    // 权值
14 } Edge;
15
16 /**
17 * 图结构体
18 */
19 typedef struct {
20     char vexs[MAX];    // 顶点集合
21     int matrix[MAX][MAX]; // 邻接矩阵
22     int vexnum;        // 顶点数
23     int edgenum;       // 边数
24 } Graph;
25
26 /**
27 * 辅助函数：根据顶点字符获取索引
28 */
29 int get_position(Graph G, char ch) {
30     for (int i = 0; i < G.vexnum; i++) {
31         if (G.vexs[i] == ch)
32             return i;
33     }
34     return -1;
35 }
36
37 /**
38 * 辅助函数：从邻接矩阵中提取所有的边
39 * 返回值：指向 Edge 数组的指针
40 */
41 Edge* get_edges(Graph G) {
42     int index = 0;
43     Edge* edges = (Edge*)malloc(G.edgenum * sizeof
44
45     for (int i = 0; i < G.vexnum; i++) {
46         // j 从 i+1 开始，只遍历矩阵的上三角，避免重
```

```
47         for (int j = i + 1; j < G.vexnum; j++) {
48             if (G.matrix[i][j] != INF) {
49                 edges[index].start = G.vexs[i];
50                 edges[index].end = G.vexs[j];
51                 edges[index].weight = G.matrix[i]
52                 index++;
53             }
54         }
55     }
56     return edges;
57 }
58
59 /**
60  * 辅助函数：对边数组进行排序（冒泡排序，从小到大）
61  * 贪心策略的核心：优先选择短边
62  */
63 void sort_edges(Edge* edges, int edgenum) {
64     for (int i = 0; i < edgenum - 1; i++) {
65         for (int j = 0; j < edgenum - 1 - i; j++)
66             if (edges[j].weight > edges[j + 1].weight) {
67                 Edge temp = edges[j];
68                 edges[j] = edges[j + 1];
69                 edges[j + 1] = temp;
70             }
71     }
72 }
73 }
74
75 /**
76  * 核心辅助函数：查找顶点的“终点”（所在集合的代表/根节点）
77  * 用于判断两个点是否已经在同一个连通分量中
78  */
79 int get_end(int vends[], int i) {
80     // 只要当前点指向的不是自己，就说明它还有上级，继续找
81     while (vends[i] != i) {
82         i = vends[i];
83     }
84     return i;
85 }
86
87 /**
88  * Kruskal 最小生成树算法
89  */
90 void kruskal(Graph G) {
91     int i, m, n;
92     int index = 0;          // 结果数组索引
```

```
93     int length = 0;           // 最小生成树总权值
94     int vends[MAX];           // 并查集数组：记录每个
95     Edge rets[MAX];           // 结果数组：保存最终选
96     Edge* edges;              // 所有的边
97
98     // 1. 获取图中所有的边
99     edges = get_edges(G);
100
101     // 2. 将边按照权值从小到大排序
102     sort_edges(edges, G.edgenum);
103
104     // 3. 初始化并查集
105     // 初始状态：每个顶点的终点都是它自己（互不连通）
106     for (i = 0; i < G.vexnum; i++) {
107         vends[i] = i;
108     }
109
110     // 4. 遍历每一条边（从小到大）
111     for (i = 0; i < G.edgenum; i++) {
112         // 获取这条边起止点的索引
113         int p1 = get_position(G, edges[i].start);
114         int p2 = get_position(G, edges[i].end);
115
116         // 查找 p1 和 p2 所在的集合代表（终点）
117         m = get_end(vends, p1);
118         n = get_end(vends, p2);
119
120         // 5. 判断是否形成环路
121         // 如果 m != n, 说明两个点目前在不同的集合中
122         if (m != n) {
123             vends[m] = n;           // 【合并集合】
124             rets[index++] = edges[i]; // 将这条边加入结果
125             length += edges[i].weight;
126         }
127         // 如果 m == n, 说明这两个点已经连通了，再加
128     }
129
130     // 6. 打印结果
131     printf("Kruskal 最小生成树结果（总权值=%d):\n",
132         length);
133     for (i = 0; i < index; i++) {
134         printf("(%c, %c) 权值:%d\n", rets[i].start, rets[i].end, rets[i].weight);
135     }
136
137     // 释放动态分配的内存
138     free(edges);
139 }
```

```

139
140 int main() {
141     // 构建示例图: A, B, C, D, E
142     char vexs[] = {'A', 'B', 'C', 'D', 'E'};
143     int matrix[5][5] = {
144         /* A */ {0, 2, 4, 7, INF},
145         /* B */ {2, 0, 1, INF, 5},
146         /* C */ {4, 1, 0, INF, 6},
147         /* D */ {7, INF, INF, 0, 3},
148         /* E */ {INF, 5, 6, 3, 0}
149     };
150
151     Graph G;
152     G.vexnum = 5;
153     G.edgenum = 7; // 图中有 7 条有效边
154
155     for (int i = 0; i < G.vexnum; i++) {
156         G.vexs[i] = vexs[i];
157         for (int j = 0; j < G.vexnum; j++) {
158             G.matrix[i][j] = matrix[i][j];
159         }
160     }
161
162     kruskal(G);
163
164     return 0;
165 }

```

时间复杂度:

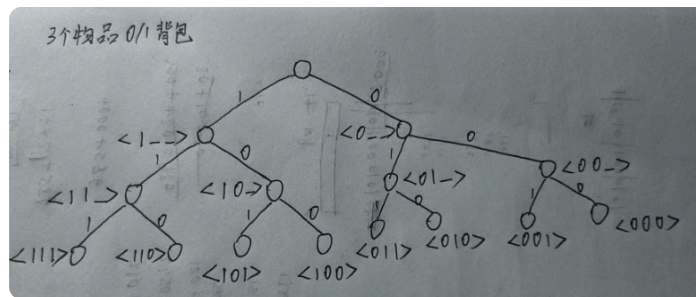
- 排序边: $O(E \log E)$
- 并查集操作: $O(E\alpha(V))$ (α 为阿克曼函数的反函数, 极小, 可视为常数)
- 总复杂度: $O(E \log E)$

回溯

0-1背包问题

解向量: $\langle x_1, x_2, \dots, x_n \rangle$ x_i 表示第 i 个物品选或不选

解空间树:



1766566630155

```

1
2 // 新增全局变量
3 int x[N];          // 当前解: x[i]=1 表示第i个物品放入, (
4 int bestx[N];      // 最优解: 用来保存最终结果
5 /**Type cw = 0;    // 当前重量
6 Type cv = 0;       // 当前价值
7 Type bestp = 0;    // 当前最优价值
8 c 背包容量
9 * 计算上界函数 (Bound)
10 * 作用: 预测当前分支理论上能达到的最高价值
11 */
12 Type Bound(int i) {
13     Type cleft = c - cw; // 剩余容量
14     Type b = cv;        // 当前已获价值
15
16     // 尽力装入完整物品
17     while (i <= n && w[i] <= cleft) {
18         cleft -= w[i];
19         b += v[i];
20         i++;
21     }
22
23     // 如果还有空间, 装入下一个物品的一部分 (分数背包退
24     if (i <= n) {
25         b += v[i] / w[i] * cleft;
26     }
27     return b;
28 }
29
30 /**
31 * 回溯核心函数
32 * i: 当前处理第 i 个物品
33 */
34
35
36 void BackTrack(int i) {
37     // 1. 达到叶子节点

```

```

38     if (i > n) {
39         bestp = cv;
40
41         // 【修改点1】：当找到最优解时，把当前的路径 >
42         for (int j = 1; j <= n; j++) {
43             bestx[j] = x[j];
44         }
45         return;
46     }
47
48     // 2. 左子树：放入物品 i
49     if (cw + w[i] <= c) {
50         x[i] = 1;        // 【修改点2】：记录当前物品
51         cw += w[i];
52         cv += v[i];
53
54         BackTrack(i + 1);
55
56         cw -= w[i];      // 回溯
57         cv -= v[i];
58         x[i] = 0;        // 【回溯】：虽然这一步不写逻辑
59     }
60
61     // 3. 右子树：不放入物品 i
62     if (Bound(i + 1) > bestp) {
63         x[i] = 0;        // 【修改点3】：记录当前物品
64         BackTrack(i + 1);
65     }
66 }

```

该算法的 最坏时间复杂度 是：

$$O(n \cdot 2^n)$$

其中：

- n 是物品的数量。
- 2^n 来自于解空间树的大小（子集树）。
- 额外的 n 系数来自于 Bound 函数的计算开销。

详细分析

我们可以从 **解空间结构** 和 **节点计算耗时** 两个方面来分析。

1. 解空间树 (Subset Tree)

0-1背包问题的解空间是一棵 **子集树**。

- 对于 n 个物品，每个物品都有 2 种选择（选 或 不选）。
- 这构成了一棵二叉树，树的深度为 n 。
- **总节点数**：满二叉树的节点数接近 2^{n+1} ，即 $O(2^n)$ 量级。

2. 单个节点的计算开销

在递归过程的每一个节点（即每次调用 BackTrack），我们可能会调用 Bound(i) 函数来计算上界。

- Bound 函数内部包含一个 while 循环，最坏情况下会扫描剩余的所有物品。
- 因此，Bound 函数的时间复杂度是 $O(n)$ 。

3. 综合复杂度

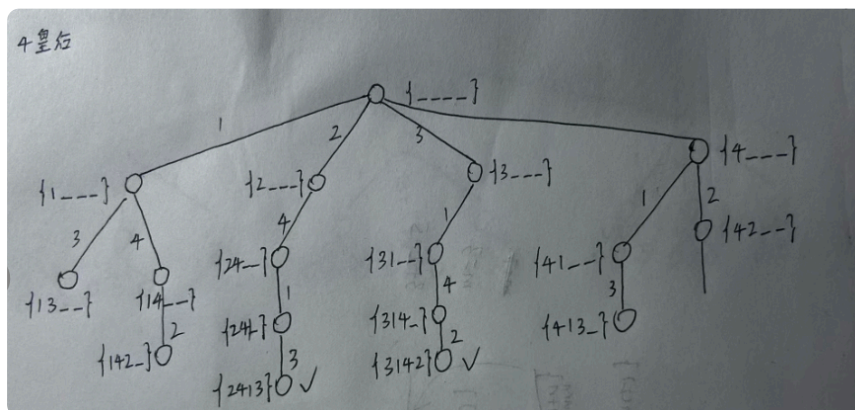
- **最坏情况**：如果你的剪枝策略（Bound 函数）失效（例如数据构造得很特殊，使得每次计算出的上界都比 bestp 大），算法就必须遍历整棵树。
- 计算公式：总时间 $\approx$ 节点总数 $\times$ 单节点开销 $\approx 2^n \times n$

N皇后

解向量设计：给棋盘上的行和列从1到N编号，同时也给皇后1-N编号，假设皇后i放在第i行，则问题可以表示为n元组

$$[x_1, x_2, \dots, x_i, x_j, \dots, x_N]$$

x_i 表示i皇后的列号



1. **列约束**：当 $i \neq j$ 时，要求 $x_i \neq x_j$ （即不同行不能在同一列）。

2. **对角线约束**: 要求 $|i - j| \neq |x_i - x_j|$ (即不能在同一对角线上)。

```

1  // =====
2  // 1. 约束函数 Place (与迭代版完全一样)
3  // =====
4  bool Place(int k) {
5      for (int i = 1; i < k; i++) {
6          // 检查: 是否同列 || 是否同对角线
7          if ((x[i] == x[k]) || (abs(i - k) == abs(x[i] - x[k])))
8              return false;
9      }
10 }
11 return true;
12 }
13
14 // =====
15 // 2. 递归回溯函数 Backtrack
16 //    t 代表当前正在处理第 t 行
17 // =====
18 void Backtrack(int t) {
19     // --- 终止条件 (Base Case) ---
20     // 如果 t > n, 说明前 n 行都已经放好了, 找到了一个
21     if (t > n) {
22         total_solutions++;
23         // 打印这个解 (可选)
24         cout << "解 " << total_solutions << ": ";
25         for (int i = 1; i <= n; i++) cout << x[i] << " ";
26         cout << endl;
27         return;
28     }
29
30     // --- 递归步骤 (Recursive Step) ---
31     // 尝试当前行 t 的每一列 (从 1 到 n)
32     for (int j = 1; j <= n; j++) {
33         x[t] = j; // 尝试把第 t 行皇后放在第 j 列
34
35         // 如果放在这里不冲突 (满足约束)
36         if (Place(t)) {
37             Backtrack(t + 1); // 【递归】深入下一层,
38         }
39
40         // 【回溯】(Backtracking)
41         // 注意: 递归函数执行完返回这里时, 意味着 t+1
42         // 我们不需要显式写 x[t]=0, 因为下一次循环 j+
43         // 这就是递归的“自动回溯”特性。

```

```
44     }  
45 }
```

1. 理论上限（最坏情况）

如果我们使用回溯法（Backtracking）来寻找**所有解**，算法的核心逻辑是逐行放置皇后。

- **第 1 行**：有 N 种选择（即放在第 1 列到第 N 列）。
- **第 2 行**：为了避免列冲突，剩下 $N - 1$ 种选择。
- **第 3 行**：剩下 $N - 2$ 种选择。
- ...
- **第 N 行**：只剩下 1 种选择。

如果不考虑对角线的限制，仅考虑列不冲突，可能的状态总数就是 $N \times (N - 1) \times (N - 2) \times \dots \times 1 = N!$ 。

因此，算法搜索空间的上界是 $O(N!)$ 。

2. 实际运行效率（剪枝的影响）

虽然理论上界是 $O(N!)$ ，但在实际运行中，我们加入了 **对角线冲突检测**（即你的代码中的 `is_ok` 函数或布尔数组判断）。

- 一旦发现当前位置与之前的皇后在对角线上冲突，算法会立即停止该分支的搜索（剪枝/Pruning）。
- 这意味着算法根本不会遍历完整的 $N!$ 个节点。实际访问的节点数远小于 $N!$ 。
- 尽管如此，对于非常大的 N ，其增长速度仍然是指数级甚至是超指数级的。

3. 具体代码实现的细节影响

你之前提供的代码中，`is_ok` 函数的实现方式也会微调复杂度：

```
1 // 你的 is_ok 函数  
2 bool is_ok(int row) {  
3     for (int i = 0; i < row; i++) { // 这里有一个 O(  
4         if (...) return false;  
5     }  
6     return true;  
7 }
```

- **常规回溯**：如果使用布尔数组（colUsed, diag1, diag2）来判断冲突，判断操作是 $O(1)$ 的。此时总复杂度接近 $O(N!)$ 。
- **你的回溯**：由于 is_ok 内部有一个循环，判断一个位置是否合法的代价是 $O(N)$ 。因此，严格来说，你的代码的时间复杂度上界是 $O(N \cdot N!)$ 。

m图着色

代码实现

递归

```

1 void Color::Backtrack(int t)
2 {
3     if (t>n) {
4         sum++;
5         for (int i=1; i<=n; i++)
6             cout << x[i] << ' ';
7         cout << endl;
8     }
9     else{
10        for (int i=1;i<=m;i++) {
11            x[t]=i;
12            if (Ok(t)) Backtrack(t+1);
13        }
14    }
15    bool Color::Ok(int k)
16    { // 检查颜色可用性
17        for (int j=1;j<k;j++)
18            if ((a[k][j]==1)&&(x[j]==x[k])) return false;
19        return true;
20    }

```

非递归

```

1 /**
2  * 非递归 m 图着色算法
3  * n: 顶点数, m: 颜色数, a: 邻接矩阵, x: 解向量
4  */
5 void m_coloring(int n, int m, bool a[][MAXN], int
6     // 1. 初始化解向量
7     for (int i = 1; i <= n; i++) {
8         x[i] = 0; // x[i]=0 表示该顶点尚未着色

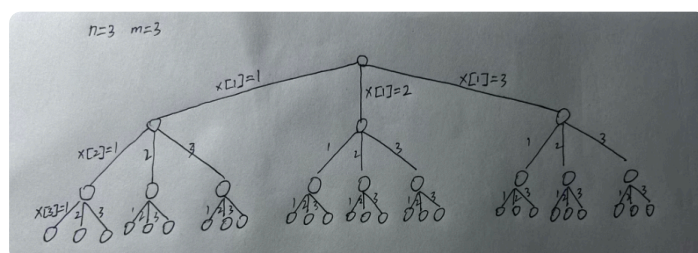
```

```

9      }
10
11     int k = 1; // 从第一个顶点开始处理
12
13     // 外层循环: 控制顶点 1, 2, ..., n 的着色过程
14     while (k >= 1) {
15         x[k] = x[k] + 1; // 为当前顶点 k 尝试下一种颜色
16
17         // 内层循环: 寻找顶点 k 的下一个可用颜色
18         while (x[k] <= m && !OK(k, a, x)) {
19             x[k] = x[k] + 1; // 如果当前颜色冲突, 尝试
20             }
21
22         // 判断是否为顶点 k 找到了合适的着色
23         if (x[k] <= m) {
24             // 成功找到颜色
25             if (k == n) {
26                 // 如果当前是最后一个顶点, 说明找到一
27                 // 输出解向量或进行处理
28                 break;
29             } else {
30                 // 如果不是最后一个顶点, 继续处理下一
31                 k = k + 1;
32             }
33         } else {
34             // 没有为顶点 k 找到合适着色 (x[k] > m)
35             x[k] = 0; // 撤销当前顶点的着色状态
36             k = k - 1; // 回退到前一个顶点, 尝试它的
37         }
38     }
39 }

```

对于解空间树中的每一个 **非叶节点**，在扩展它的子节点时，最坏情况下的耗时是 $O(mn)$ 。



1766566684681

- **m 的含义**：当前节点有 m 个分叉（对应 m 种颜色），算法需要尝试每一个子节点。

- **n 的含义**：对于这 m 个可能的颜色，每一个都要调用一次 OK 函数。OK 函数需要检查当前顶点与图中其它最多 $n - 1^{**}$ 个顶点的相邻关系及颜色冲突，耗时为 $O(n)^{**}$ 。
- **结论**：因此，处理一个父节点以产生其所有子节点所需的总时间是 $m \times O(n) = O(mn)$ 。

$$\sum_{i=0}^n m^i (mn)$$

- $\sum m^i$ ：这是解空间树中各层节点的总和。
- (mn) ：这是在每一层处理节点扩展时的单位工作量。

根据等比数列求和公式推导：

$$nm(m^n - 1)/(m - 1) = O(nm^n)$$

旅行商问题TSP

目标是寻找一条回路，使得总费用（距离）最小化

$$\min \left\{ \sum_{i=1}^{n-1} w(v_i, v_{i+1}) + w(v_n, v_1) \right\}$$

- **第一部分** $\sum_{i=1}^{n-1} w(v_i, v_{i+1})$ ：表示从起点 v_1 出发，依次经过其它 $n - 1$ 个城市，直到到达最后一个城市 v_n 的路径总长。
- **第二部分** $w(v_n, v_1)$ ：表示从最后一个城市 v_n 回到起点 v_1 的距离（闭合回路）。
- $w(v_i, v_j)$ 或 $w(i, j)^{****}$ ：表示城市 i 和城市 j 之间的直接距离（权重）。
- $w(v_i, v_j) = \infty$ ：表示两个城市之间没有直接路径（非完全图的情况）。
- **顶点的编号**：将 n 个城市简单编号为 $1, 2, \dots, n$ 。
- **解空间大小** $= (n - 1)!^{****}$
- **推导逻辑**：虽然总共有 n 个城市，全排列是 $n!$ ，但由于TSP是回路问题，起点的选择不影响路径的总长度（是一个圈）。为了消除重复计算，我们固定起点（如节点1），剩下的 $n - 1$ 个节点进行全排列。

剪枝

剪枝/约束条件 1: 连通性剪枝 (Feasibility)

如果当前正在考虑的顶点 j , 与当前已经走过的部分路径中的末端结点 i 没有边相连 (即 $w[i, j] = \infty$), 则放弃搜索 j 所在分支。

剪枝/约束条件 2: 最优性剪枝 (Optimality)

令到第 i 层结点为止, 构造的部分解路径为 $\langle x_1, x_2, \dots, x_i \rangle$

该路径的当前权值总和 cw (Current Weight) 为:

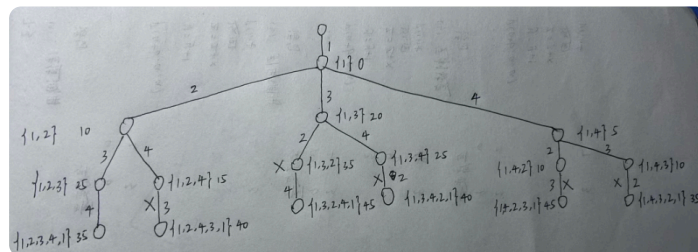
$$cw = \sum_{j=1}^{i-1} w(x_j, x_{j+1})$$

剪枝逻辑:

如果 当前部分路径的开销 cw 已经大于或等于 当前已找到的最优解 (Best Weight, $bestw$), 则剪枝。

- 为什么剪枝? 即使剩下的路径长度为0 (不可能), 这条路的总长度也已经超过了之前找到的一条完整回路 (最优解)。继续往下搜只会得到更差的结果, 纯属浪费时间。

解空间树



1766565547188

代码实现

```

1 // 假设最大城市数量 N
2 #define N 100
3 #define INF 99999999 // 表示无穷大, 即无通路
4
5 int n; // 城市数量
6 int w[N][N]; // 邻接矩阵, w[i][j] 表示城市 i
7 int x[N]; // 当前搜索过程中的部分路径 (解
8 int bestx[N]; // 记录当前找到的最优路径
9 int cw = 0; // (Current Weight) 当前已经走

```

```

10  int bestw = INF;      // (Best Weight) 当前找到的最优
11
12
13
14  void BacktrackTSP(int i) {
15      // =====
16      // 1. 叶子节点处理：到达递归底层 (i == n)
17      // =====
18      if (i == n) {
19          // 检查两个条件：
20          // 1. 倒数第二个城市 x[n-1] 到 最后一个城市 x[n] 是否有路
21          // 2. 最后一个城市 x[n] 到 起点 1 是否有路
22          if (w[x[n-1]][x[n]] != INF && w[x[n]][1] != INF) {
23
24              // 判断当前完整回路费用是否比已知的最优解小
25              if (cw + w[x[n-1]][x[n]] + w[x[n]][1] < bestw) {
26
27                  // 更新最优权值
28                  bestw = cw + w[x[n-1]][x[n]] + w[x[n]][1];
29
30                  // 记录当前最优路径
31                  for (int j = 1; j <= n; j++) {
32                      bestx[j] = x[j];
33                  }
34              }
35          }
36          return;
37      }
38
39      // =====
40      // 2. 递归扩展子树：遍历当前层所有可能的选择
41      // =====
42      else {
43          // 尝试每一个还未访问的城市（通过交换位置实现排列）
44          for (int j = i; j <= n; j++) {
45
46              // --- 剪枝 / 约束条件检查 ---
47              // 1. 连通性剪枝：x[i-1] 到 待选城市 x[j] 是否有路
48              // 2. 最优性剪枝：当前花费 + 新的一步花费 < 当前最优解
49              if (w[x[i-1]][x[j]] != INF && (cw + w[x[i-1]][x[j]] + w[x[j]][1] < bestw)) {
50
51                  // [前进]：准备搜索下一层
52                  swap(x[i], x[j]);          // 1.
53                  cw += w[x[i-1]][x[i]];      // 2.
54
55                  // [递归]：深度优先搜索下一层

```

```
56         BacktrackTSP(i + 1);           //
57
58         // [回溯]: 恢复状态, 准备尝试 loop 中
59         cw -= w[x[i-1]][x[i]];         // 1.
60         swap(x[i], x[j]);              // 2.
61     }
62 }
63 }
64 }
```

时间复杂度

$O(n!)$

(注: 更精确的说法是 $O((n - 1)!)$, 但在大O表示法中, 通常简写为阶乘级 $O(n!)$)